Vijay Academy Presents

Savitribai Phule Pune University (SPPU)

Web Technology (WT)

TE Computer Engineering (2019 Pattern)

One Shot Lecture

Unit 4. JSP and Web Services

What You Get : Strategy For Exam :

- Simple Explanation
- PPT + Notes
- PYQs + Most IMP Questions

Subscribe The Vijay Academy For Free Engineering Content

Unit 4. JSP and Web Services

Syllabus Topics :

JSP Topics :

1. Introduction to JSP & JSP vs Servlets
2. JSP Lifecycle
3. Basic JSP (Scriptlet, Expression, Declaration, Comment, Directives)
4. JSP Implicit Objects
5. JavaBeans Classes & JSP
6. JSP Support for MVC (Model-View-Controller)
7. JSP Related Technologies

Web Services Topics :

8. Web Service Concepts & Benefits
9. Writing a Java Web Service & Client
10. WSDL (Web Services Description Language)
11. SOAP (Simple Object Access Protocol)

Struts Topics :

12. Struts Overview & Architecture
13. Struts Actions & Interceptors
14. Struts Result Types & Validations
15. Struts Localization, Exception Handling & Annotations

1. Introduction to JSP & JSP vs Servlets

Definition :

JSP (Java Server Pages) is a technology used to create **dynamic web pages** using HTML + Java code together. Instead of writing all HTML inside Java (like Servlet), JSP lets you write **HTML directly** with small Java code embedded in it.

Example :

```
<html>
<body>
  <h2>Welcome!</h2>
  <%
    String name = "Vijay";
    out.println("Hello, " + name);
  %>
</body>
</html>
```

Normal HTML is written directly
<% %> tag contains Java code
out.println() prints output on page
Server runs this and sends result to browser

1. Introduction to JSP & JSP vs Servlets

Key Points :

JSP runs on the **server side.**

JSP is built **on top of Servlets.**

JSP separates **presentation (HTML)** from **business logic (Java)**

JSP files have extension **.jsp**

JSP is easier to write than Servlet for web pages.

JSP vs Servlets

Feature	JSP	Servlet
Code style	HTML + Java	Pure Java
Easy to write UI	Yes	No
Compilation	Auto (by server)	Manual
Best for	Presentation layer	Business logic
Maintenance	Easy	Difficult
Extension	.jsp	.java

2. JSP Lifecycle

Definition

JSP Lifecycle means the step-by-step process of what happens from the time a JSP page is **requested** by user to the time **response is sent** back.

Key Points

JSP lifecycle is similar to Servlet lifecycle

JSP first gets **translated into a Servlet** by the server

Then it goes through **compilation → loading → initialization → execution → destroy**

This translation happens **only once** (first request)

From second request onwards — directly executed (faster!)

Lifecycle Methods :

Method	When Called	Purpose
jspInit()	Once at start	Initialize resources
_jspService()	Every request	Process request & send response
jspDestroy()	Once at end	Cleanup resources

JSP Lifecycle Diagram :

User Request (first time)

↓

1. Translation → .jsp file converts to .java (Servlet)

↓

2. Compilation → .java compiles to .class file

↓

3. Loading → Class loaded into memory

↓

4. Instantiation → Object of Servlet created → jspInit() called (once only)

↓

5. Execution → _jspService() called (every request)

↓

6. Destruction → jspDestroy() called (server shutdown)

↓

Response sent to User

3. Basic JSP Tags

Definition

JSP provides special **tags** to Java code inside HTML page.
These tags tell the server — *"this part is Java code, execute it!"*

Types of JSP Tags :

1. Scriptlet Tag <% %>

Used to write **Java statements** inside JSP.

Example :

```
<%
  int a = 10;
  int b = 20;
  out.println("Sum = " + (a+b));
%>
```

Output: Sum = 30

2. Expression Tag <%= %>

Used to **print/display** a value directly. No need of out.println().

Example :

```
<%= "Hello World" %>
<%= 5 + 3 %>
```

Output: Hello World and 8

3. Declaration Tag <%! %>

Used to declare **variables or methods** that belong to the whole JSP class.

Example :

```
<%!
  int count = 0;

  public int square(int n) {
    return n * n;
  }
%>
<%= square(5) %>
```

Output: 25

4. Comment Tag <%-- --%>

Used to write **comments** in JSP. These are NOT sent to browser.

Example :

<%-- This is a JSP comment, not visible in browser --%>

<!-- This is HTML comment, visible in page source -->

5. Directive Tag <%@ %>

Used to give **instructions to the server** about the JSP page.

3 types of Directives:

Directive	Syntax	Purpose
page	<%@ page %>	Set page properties
include	<%@ include %>	Include another file
taglib	<%@ taglib %>	Use custom tag library

Example :

```
<%@ page language="java" contentType="text/html" %>
<%@ page import="java.util.*" %>
<%@ include file="header.jsp" %>
```

Full Example Using All Tags

```
<%@ page language="java" contentType="text/html" %>
<html>
<body>

<%-- This is a JSP comment --%>

<%!
  String greet(String name) {
    return "Hello, " + name + "!";
  }
%>

<%
  String user = "Vijay";
%>

<h2><%= greet(user) %></h2>

</body>
</html>
```

Output: Hello, Vijay!

4. JSP Implicit Objects

Definition

Implicit Objects are ready-made objects in JSP that are **automatically created by the server**. You don't need to create them — just use them directly!

Think of them as **free tools** available in every JSP page.

Key Points :

There are **9 implicit objects** in JSP

They are created by the **JSP container automatically**

Available **directly** inside scriptlet <% %> tags

No need to write new keyword to create them

4. JSP Implicit Objects

All 9 Implicit Objects :

Object	Type	Purpose
out	JspWriter	Print output to browser
request	HttpServletRequest	Get data from user/form
response	HttpServletResponse	Send response to browser
session	HttpSession	Store user session data
application	ServletContext	Share data across whole app
config	ServletConfig	Get servlet config info
pageContext	PageContext	Access all other objects
page	Object	Current JSP page itself
exception	Throwable	Handle errors/exceptions

5. JavaBeans Classes & JSP

Definition :

JavaBean is a simple Java class that follows some rules.

In JSP, we use JavaBeans to **separate Java logic from HTML** — keeping the page clean and organized.

Think of JavaBean as a **data container** — it stores and gives back data.

Rules of JavaBean :

Must have a **public no-argument constructor**

All variables must be **private**

Must have **getter and setter** methods.

Key Points :

JavaBeans keep **business logic separate** from JSP page

JSP uses special tags to work with JavaBeans

Makes code **reusable and clean**

Bean properties are accessed using **getter/setter methods.**

5. JavaBeans Classes & JSP

3 JSP Tags for JavaBeans

Tag	Purpose
<jsp:useBean>	Create or find a Bean object
<jsp:setProperty>	Set value into Bean
<jsp:getProperty>	Get value from Bean

Example :

Bean Class (Student.java)

```
public class Student {
  private String name;  // private variable

  public Student() {}   // no-arg constructor

  // setter - set value
  public void setName(String name) {
    this.name = name;
  }

  // getter - get value
  public String getName() {
    return name;
  }
}
```

Use in JSP :

```
<jsp:useBean id="s" class="Student" scope="session"/>
<jsp:setProperty name="s" property="name" value="Vijay"/>

Name is: <jsp:getProperty name="s" property="name"/>
```

Output :

Name is : Vijay

6. JSP Support for MVC (Model-View-Controller)

Definition :

MVC is a design pattern that **divides a web application into 3 parts** so that code is clean, organized and easy to manage.

M — Model → Data & Business Logic

V — View → What user sees (UI)

C — Controller → Handles requests & connects M and V

Key Points :

MVC separates **data, display and control** logic

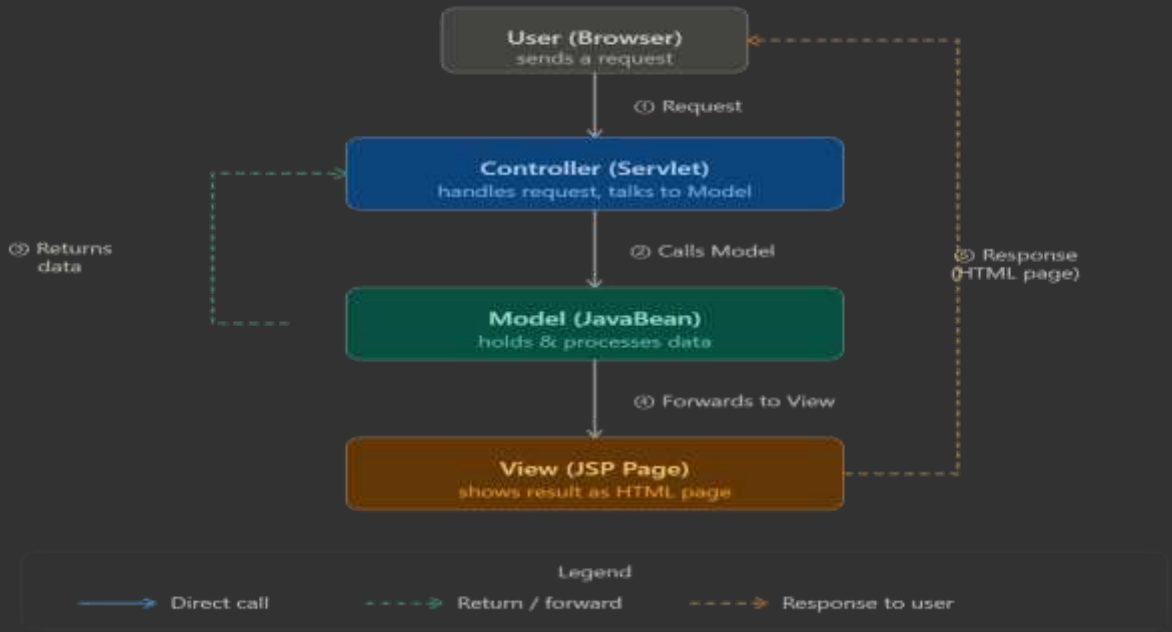
In JSP based MVC:

- **Model** = JavaBean / Java class
- **View** = JSP page
- **Controller** = Servlet

Makes application **easy to maintain**

Changes in UI don't affect business logic.

MVC Diagram :



Explanation :

Step ① — User types URL in browser and sends a **Request** to the Controller (Servlet).

Step ② — Controller **calls the Model** (JavaBean) to get or process data.

Step ③ — Model **returns the data** back to the Controller (e.g. student name, marks).

Step ④ — Controller **forwards** the data to the View (JSP page).

Step ⑤ — View creates an HTML page and sends the **Response** back to the user's browser.

7. JSP Related Technologies

Definition

JSP Related Technologies are the **other tools and technologies** that work together with JSP to build complete web applications.

Key Technologies

1. Servlets

JSP is built **on top of Servlets.**

JSP pages internally get **converted to Servlets.**

Servlets handle **business logic**, JSP handles **display.**

They work together in MVC pattern.

2. JavaBeans

Reusable **Java components** used with JSP

Store and manage **data** between JSP pages

Used with useBean, setProperty, getProperty tags

7. JSP Related Technologies

3. JSTL (JSP Standard Tag Library)

A set of **ready-made tags** for JSP

Replaces Java code inside JSP with simple tags

Makes JSP pages **cleaner and easier** to read

4. EL (Expression Language)

Simple way to **display data** in JSP

Uses ${ } syntax

Cleaner and simpler than scriptlets.

5. Filters

Code that runs **before or after** a servlet/JSP

Used for **authentication, logging, compression**

Like a security guard before entering JSP.

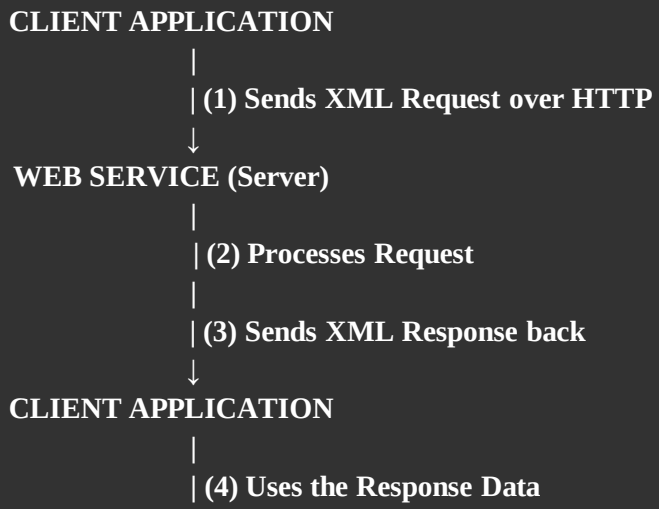
8. Web Service Concepts & Benefits

A **Web Service** is a way for two different applications to **communicate with each other over the internet.** — even if they are built in different programming languages or run on different platforms.
Think of it like a **translator** between two people who speak different languages.

Key Points :

Web Services use **HTTP** to communicate.
Data is exchanged in **XML or JSON** format.
Platform independent — Java app can talk to Python app.
Uses standard protocols like **SOAP, WSDL, UDDI.**
Two types: **SOAP-based** and **REST-based.**

How Web Service Works :



Example :

Imagine a **Weather Web Service:**

Mobile App (Java) ⟶ Weather Web Service ⟶ Returns temperature data

Website (PHP) ⟶ Same Web Service ⟶ Returns same data

Desktop App (.NET)⟶ Same Web Service ⟶ Returns same data

All three different apps use the **same web service** — this is reusability!

Benefits of Web Services :

Benefit	Explanation
Platform Independent	Java, Python, .NET can all communicate
Language Independent	Any programming language can use it
Reusability	One service used by many applications
Interoperability	Different systems work together
Loosely Coupled	Changes in one app don't break other
Uses HTTP	Works over standard internet protocol

Key Components of Web Service

1. SOAP

Protocol for **sending messages** between applications

Messages are in **XML format**

Full form: Simple Object Access Protocol

2. WSDL

Describes **what the web service does**

Like a **menu card** of available services

Full form: Web Services Description Language

3. UDDI

A **directory** where web services are registered

Used to **find/discover** web services

Full form: Universal Description, Discovery & Integration

9. Writing a Java Web Service & Client

Definition :

In Java, we can create a **Web Service** using simple annotations.

The @WebService annotation tells Java — *"this class is a web service!"*

A **Web Service Client** is a program that **calls and uses** the web service.

Key Annotations Used

Annotation	Meaning
@WebService	Marks class as a Web Service
@WebMethod	Marks method as available to clients
@WebParam	Names the parameter in WSDL

10. WSDL (Web Services Description Language)

Definition :

WSDL is an **XML-based file** that describes a web service — it tells the client:

What the service does (methods available)

How to call it (input/output format)

Where to find it (URL/address)

Think of WSDL as a **menu card of a restaurant** — it tells you what is available and how to order it.

Key Points :

WSDL stands for **Web Services Description Language**

It is written in **XML format**

Auto-generated by Java when you publish a web service

Available at: http://yourservice?wsdl

Client reads WSDL to understand how to use the service

10. WSDL (Web Services Description Language)

Main Elements of WSDL

Element	Purpose
<types>	Defines data types used
<message>	Defines input/output messages
<portType>	Defines operations (methods) available
<binding>	Defines communication protocol (SOAP/HTTP)
<service>	Defines the address/URL of service

WSDL Structure:

```
<definitions name="CalculatorService">

  <!-- WHAT data is used -->
  <types>
    input: two integers (a, b)
    output: one integer (result)
  </types>

  <!-- WHAT messages are sent -->
  <message name="addRequest">
    send: a=3, b=5
  </message>
  <message name="addResponse">
    receive: result=8
  </message>

  <!-- WHAT methods are available -->
    <portType name="CalculatorPort">
      <operation name="add">
        <input>  addRequest  </input>
        <output> addResponse </output>
      </operation>
    </portType>

    <!-- HOW to communicate -->
    <binding> use SOAP over HTTP </binding>

    <!-- WHERE the service is -->
    <service>
      http://localhost:8080/CalculatorService
    </service>

</definitions>
```

11. SOAP (Simple Object Access Protocol)

Definition :

SOAP is a **protocol (set of rules)** for sending messages between a client and a web service over the internet.
SOAP messages are written in **XML format** and travel over **HTTP**.
Think of SOAP as a **standard envelope** — just like how a letter has a fixed format (address, stamp, content), SOAP has a fixed format for sending data.

Key Points :

SOAP stands for **Simple Object Access Protocol**
Messages are in **XML format**
Works over **HTTP, SMTP** protocols
Platform & language independent
More **secure and strict** than REST
Used in **enterprise/banking applications.**

11. SOAP (Simple Object Access Protocol)

SOAP Message Structure

A SOAP message has **4 parts:**

SOAP Message
├── Envelope → outer wrapper (mandatory)
│ ├── Header → extra info like auth (optional)
│ └── Body → actual request/response (mandatory)
│ └── Fault → error info (if any error occurs)

SOAP Request Example :

Client sends this XML to call add(3, 5):

```
<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

  <!-- Optional Header -->
  <soap:Header>
    <auth>mySecretToken123</auth>
  </soap:Header>

  <!-- Mandatory Body -->
  <soap:Body>
    <add>
      <a>3</a>
      <b>5</b>
    </add>
  </soap:Body>

</soap:Envelope>
```

UDDI :

Definition

UDDI stands for **Universal Description, Discovery and Integration.**

UDDI is like a **yellow pages directory** for web services

— businesses **register** their web services here and clients can **search and find** them.

Key Points

UDDI = **directory/registry** of web services

Businesses **publish** their services in UDDI

Clients **discover** services using UDDI

UDDI stores **WSDL link** of each service

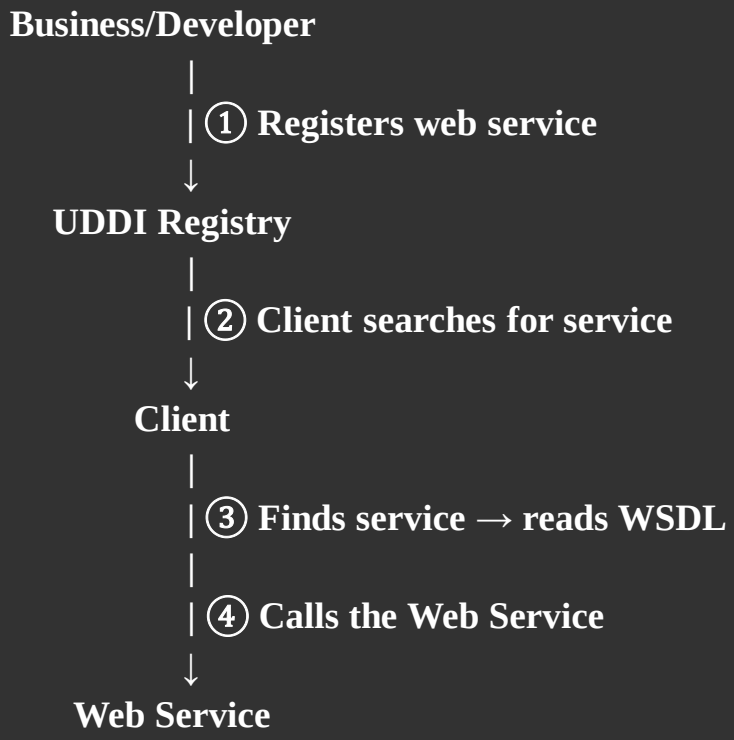
Works together with **SOAP + WSDL**

What UDDI Stores

Info	Description
White Pages	Business name, address, contact
Yellow Pages	Category of service (banking, weather)
Green Pages	Technical details (WSDL URL, how to call)

UDDI :

How UDDI Works :



12. Struts Overview & Architecture

Definition :

Struts is a **framework** for building Java web applications. It is based on the **MVC pattern** and makes web development easier and more organized.

Think of Struts as a **ready-made structure**

— instead of building everything from scratch, Struts gives you a pre-built skeleton to work with.

Key Points

Struts is an **open-source** Java framework

Based on **MVC (Model-View-Controller)** pattern

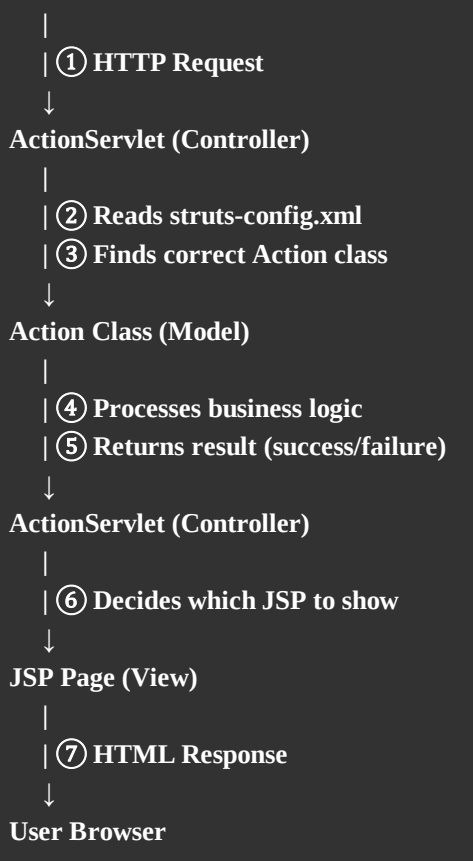
Developed by **Apache Software Foundation**

Uses **XML configuration** files

Reduces code duplication

Makes large applications **easy to manage.**

Struts Architecture Diagram :



Main Components of Struts

1. ActionServlet

The **main controller** of Struts.
Receives ALL requests from browser.
Also called **Front Controller.**
Reads struts-config.xml to decide what to do.

2. Action Class

Contains **business logic.**
Processes user request.
Returns a result string like "success" or "error".

3ActionForm

A **JavaBean** that holds form data. Automatically fills data from HTML form.

4. struts-config.xml

The **configuration file** of Struts.
Maps requests to Action classes.
Maps results to JSP pages.

5. JSP Pages (View)

Display results to user.
welcome.jsp shown on success.
error.jsp shown on failure.

13. Struts Actions & Interceptors

Struts Actions :

A **Struts Action** is a Java class that contains the **business logic** of your application.
It processes the user's request and returns a result.

Key Points :

Every user request is handled by an **Action class.**
Action class has an **execute()** method.
execute() returns a **String result** like "success", "error"
Result string decides **which JSP to show.**
Configured in struts.xml

13. Struts Actions & Interceptors

Struts Interceptors :

An **Interceptor** is a piece of code that runs **before or after** an Action class executes.

Think of interceptor as a **security check at airport** — before you board the plane (Action), you go through security (Interceptor).

Key Points :

Interceptors run **before and after** Action execution

Used for **logging, validation, authentication, exception handling**

Struts has many **built-in interceptors**

Can create **custom interceptors** too

Configured in struts.xml

14. Struts Result Types & Validations

Result Types :

Result Types in Struts decide **what happens after** an Action executes.
— which page to show, where to redirect or what data to send back.

Key Points :

Action returns a **result string** like "success", "error"
Result type decides **how** to handle that result
Different result types for different needs
Configured in struts.xml

Important Result Types :

Result Type	Purpose
dispatcher	Forward to JSP page (default) - Show result page
redirect	Redirect to new URL
redirectAction	Redirect to another Action
stream	Send file/image as response
json	Send JSON data
plainText	Send plain text

14. Struts Result Types & Validations

Struts Validations :

Validation in Struts checks if the user has entered **correct data** in a form — before the Action processes it.
Think of it as a **form checker** — if username is empty or password is too short, Struts shows an error automatically.

Key Points :
Struts has **built-in validation** support.
Two ways to validate:

- **XML based** — using ActionName-validation.xml
- **Annotation based** — using @Validate annotations

Validation runs **before** Action executes.

15. Struts Localization, Exception Handling & Annotations

Localization :

Localization means showing your web application in **different languages** based on the user's location or preference.
Example: Same app shows **English** for US users and **Marathi** for Maharashtra users !

Key Points :

Struts supports localization using **property files**
Each language has its own .properties file
Struts automatically picks correct file based on user's locale
No need to change Java code for different languages

15. Struts Localization, Exception Handling & Annotations

Exception Handling :

Exception Handling in Struts means managing errors that occur during Action execution — automatically redirecting to an error page without crashing the app.

Key Points :

Struts handles exceptions using exception interceptor.

Define exception handling in struts.xml

When exception occurs → go to specific error JSP.

Two levels:

- **Action level** → handles exception for one action.
- **Global level** → handles exception for all actions.

15. Struts Localization, Exception Handling & Annotations

Annotations in Struts :

Annotations are a way to configure Struts using **Java code directly** — instead of writing XML configuration files.
Instead of editing struts.xml, you write @Result, @Action etc. directly in Java class.

Key Points :
Annotations **replace XML configuration.**
Code becomes cleaner and easier to read.
Common annotations:

- @Action → maps URL to action
- @Result → defines result pages
- @InterceptorRef → adds interceptor
- @Validation → enables validation

Thank You!

Like, Share & Subscribe

https://youtube.com/@thevijayacademy?si=uYwlDuAxyZfLsiei

Also join Telegram channel for updates and notes :

https://t.me/the_vijay_academy